

BROWSER GAME ENGINE

Technical Architecture Audit & Target Design

Source: RPG_Engine (MythMill) v1.1.0

Phase 0 Audit + Target Architecture + Migration Roadmap

Lead Engine Architect / Technical Director

READ → UNDERSTAND → MAP → AUDIT → DESIGN → MIGRATE

Full Markdown companion: [docs/Browser_Game_Engine_Architecture_Audit.md](#)

A. Executive Summary

RPG_Engine is a JSON-driven text/IF RPG engine with a large visual Editor. Strengths: Quest v2 (QuestRuntime + questProgress as source of truth), ConditionSystem, Action Registry, scene components, editor hooks contract, export path, ~173 JS modules, quest tests baseline 138 passed.

It is NOT a general 2D/3D engine: no spatial ECS, no sprite/tilemap/GLTF pipeline, no physics, no isolated Play Mode. Editor depends on global Editor object, DOM, and historically suffered recursion from monkey-patches and inspector bugs.

Verdict

Feasible to evolve into a compact multi-mode Browser Game Engine IF Core Runtime is extracted first and Editor becomes a client of Core. Do not grow 3D by further patching editor.html. Reuse QuestRuntime, Conditions, Actions, migrations, no-code UX. Replace global monkey-patches, DOM-as-runtime coupling, and flat “scene text only” world model.

B. Current Architecture Map

Layers: Author → Editor (global + hooks + DOM) → Project JSON → GameEngine + SceneManager + Components + Actions → QuestEventBus → QuestRuntime; parallel ConditionSystem, Combat, Inventory; ui-renderer DOM; Save/Load + export.

System verdicts (summary)

System	Verdict	Notes
QuestRuntime / Tasks / Events	KEEP	SoT; tested
Conditions / Actions v3	KEEP	Namespace later
Scene Manager + text scenes	EXTRACT	Become TextScene
ui-renderer	REFACTOR	Split GameUI
Character Creator / Combat	KEEP	Text/Hybrid
Editor.hooks	KEEP	Hard contract
Global Editor assign	REPLACE	Load-order bugs
EditorHistory wrap	REFACTOR	Commands
Inspector	REFACTOR	Registry-only
Export/bundles	REPLACE	Real dist pipeline
2D/3D/Physics/Assets GUID	NEW	Absent today

Incident classes to never reintroduce

- Inspector codeFieldIfAdvanced self-recursion; Editor.hooks + late Editor.foo= wrappers; questStages as SoT; campaign IDs in generic UI; full rerender per keystroke.

C. Target Architecture

Browser Game Engine

- |— Engine Core (App, EventBus, Project)
- |— Runtime (Scene, Entity, Component, Input, Audio,
| Save, Script, Asset Runtime, Game UI)
- |— Rendering (Canvas2D | WebGL2 | WebGPU backends)
- |— Physics (optional 2D/3D)
- |— Asset Pipeline (GUID, import, cache, deps)
- |— Editor (optional at runtime of shipped game)
- |— Export/Build → dist/engine.js + game.js + assets + data

Principles: Editor not required to run builds; Game UI ≠ Editor UI; one Entity/Scene foundation with mode-specific components; no monkey-patch extensions; core unit-testable without DOM; small-team complexity budget.

Entity model

Scene → Entity (guid) → Components (Transform2D/3D optional, Sprite/Mesh, Collider, Script, Text/Dialogue, domain links). Prefab v1: instantiate + overrides; nested prefabs later. Text entities may omit spatial transform.

Renderer

IRenderer abstraction; MVP backends Canvas2D (2D) and WebGL2 (3D). WebGPU when abstraction stable. Text/UI may use DOM or canvas UI pack per project.

Play Mode

EDIT → snapshot → PLAY on runtime clone → STOP disposes runtime → Editor restored. Gameplay must not mutate editor project state.

D. Migration Map (pattern OLD→Adapter→NEW→DELETE)

KEEP: QuestRuntime, tasks, events, migrate, Conditions, Actions, Character Creator, combat core, Editor.hooks contract, templates for text. ADAPTER: current scene JSON → TextScene. REFACTOR: ui-renderer, Inspector, History. REPLACE: global Editor shell, export pipeline. NEW: ECS core, assets GUID, render backends, physics optional. DELETE after cutover: monolith drift, .bak quests, dead flag progress paths.

F. Four Game Modes (system matrix)

Core/Events/ECS/Quests/Conditions/Actions/GameUI: all modes. Canvas2D/tilemap: 2D (+hybrid). WebGL2/GLTF: 3D (+hybrid). Dialogue/IF/CC: Text (+hybrid). Physics: optional non-text. Spatial viewport: 2D/3D/Hybrid; Text uses flow/graph editors.

G. Roadmap

Phase 0 Audit: This report; no mass rewrite.

Phase 1 Architecture: Packages core/runtime/render/editor/build; import boundary tests.

Phase 2 Core Runtime: EngineApp, EventBus, Entity, Component, Scene; headless tests.

Phase 3 Assets: GUID registry, load by ref, image/audio/json.

Phase 4 Renderer: IRenderer + Canvas2D sprite; WebGL2 stub.

Phase 5 Editor Foundation: Browser, hierarchy, inspector host, viewport host; no new monkey-patches.

Phase 6 Play Mode: Isolated runtime; Stop restores snapshot.

Phase 7 2D: Sprite, camera, tilemap sample; exported run.

Phase 8 3D: GLTF, light, camera; exported run.

Phase 9 Text: Adapter for existing scenes/quests; golden quest tests.

Phase 10 Hybrid: Composite scene; dialogue overlay on world.

Phase 11 Prefab/Anim: Prefab v1; sprite anim; optional 3D clips.

Phase 12 Build/Export: Hashed dist/; drop stale monolith.

Phase 13 Testing: ECS, assets, play, quests, editor boot.

Phase 14 Cleanup: Remove adapters/dead code; docs.

Phase 15 Stable: engineVersion 2.x; four mode samples.

Every change: Change → Test → Verify → Commit → Next. Spike Phase 9 adapter early to de-risk content.

I. Risk Register

Scope explosion → phase gates, TEXT+Core first. Editor recursion → hooks tests, ban monkey-patch. Quest/save break → keep QuestRuntime + migration suite. Bundle drift → single build. DOM UI forever → Game UI interface early. Nested prefabs/WebGPU early → defer. Giant PRs → small commits.

J. Definition of Done

Four templates open; Play/Stop safe; dist runs without Editor; TEXT demo + quest tests green; 2D move/trigger sample; 3D GLTF sample; Hybrid dialogue overlay; GUID assets; no Runtime→Editor imports in prod; Behaviour + Actions documented; risks closed or owned.

Recommendation: Accept Phase 0; implement Core ECS (Phases 1-2) in parallel tree; early TEXT adapter spike; then spatial renderers. One engine → one Editor → four modes → one Runtime → shared Asset/Scene/Entity foundation. RPG_Engine is the migration base, not the ceiling.